

NO es una metodología de software, sirve para desarrollar software

Forma de construir código

"El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores...El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo"

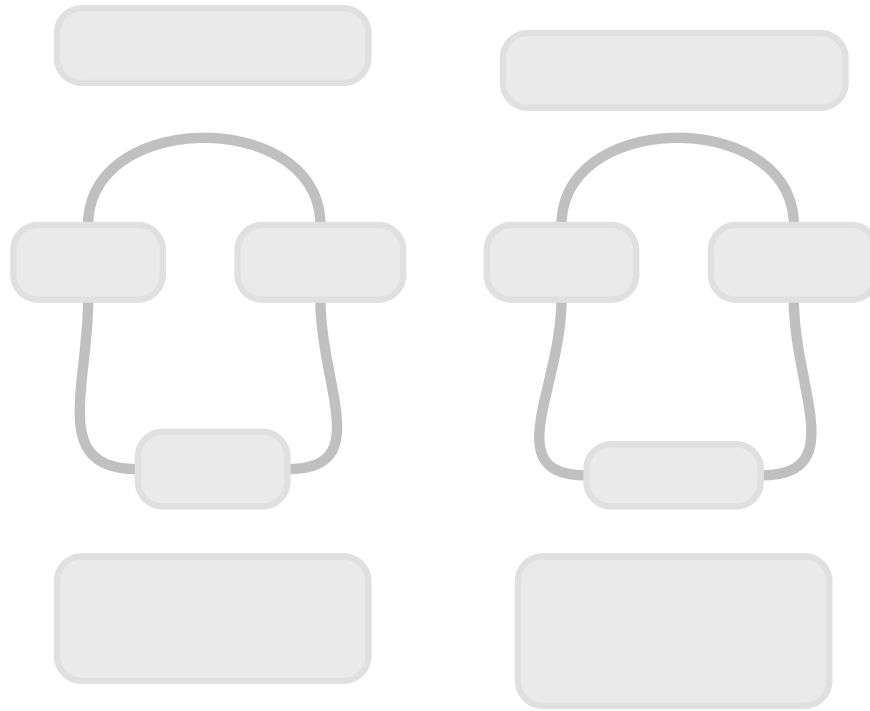
Este desarrollo se encuentra en la fase de implementación del software, no en la de testing o prueba, por más que el nombre empiece con Test

En el desarrollo tradicional, la etapa de testing, de existir, se encuentra al final de una lista de prioridades

- Captura de requerimientos
- Análisis de requerimientos
- Diseño del software
- Desarrollo del software
- Testing
- Implementación
- Mantenimiento

En el desarrollo tradicional, la idea de aseguramiento y control de calidad tiene un enfoque más reactivo, dejando que los errores ocurran en fases más tardes del proyecto, provocando que su solución implique muchos costos, tanto económicos, temporales, de confianza, etc.

Desarrollo Tradicional vs TDD



Si yo resuelvo como probar esto, voy a tener que entender como funciona el negocio

TDD empezó pensado para pruebas unitarias y posteriormente fue evolucionado a otras técnicas, como desarrollo manejado por comportamiento o por desarrollo horizontal

Con TDD primero pensamos y desarrollamos las pruebas, refactorizamos y posteriormente programamos un código que sea capaz de pasar esa prueba

Beneficios de TDD

- Nos da un código más robusto
 - Es más predecible y limpio, gracias al test podemos saber cuando ese código está listo
 - Más tolerante al cambio, si el test pasa, el cambio que hicimos es correcto, si no, hay algún error
 - Más seguro, las pruebas que se realizan pueden hacerse con mocks, por tanto no afectan al programa.
 - Tenemos menos código duplicado, ya que si la prueba pasó, es que el código existe en algún lado
- Es más económico de mantener, facilita la comprensión del código. Si no entendemos el código, podemos empezar por ver QUE debería devolver el código
- Aceleramos la velocidad de desarrollo posibilitando el despliegue continuo, dado que todo funciona

- **Asegura la cobertura de prueba?**
- Obliga a pensar como usar un componente y que endpoints hacer, por lo que lleva a pensar en una API
- ALTA COHESIÓN Y BAJO ACOPLAMIENTO

SE basa en un ciclo RED-GREEN-REFACTOR

El primer test siempre debería fallar, para que de esa manera mejoremos el código a partir de él

3 leyes fundamentales

- No escribir una línea de código hasta que no hayamos desarrollado un test que falle
- No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.
- No escribir más código que el explícita mente necesario para hacer que el test pase

Pasos para TDD

1. Escribir un test
2. Hacer que el test compile
3. Hacer que el test falle (Red)
4. Hacer un cambio para que pase el test
5. Correr el test para que pase (Green)
6. Quitar duplicación (Re-factor)

Refactoring

Forma de desarrollar que apunta a mejorar la calidad interna del componente que construimos, **NO CAMBIAMOS FUNCIONALIDAD**, solamente queremos que lo haga mejor. Cambiamos el interior sin modificar lo exterior.

Necesidad de refactoring

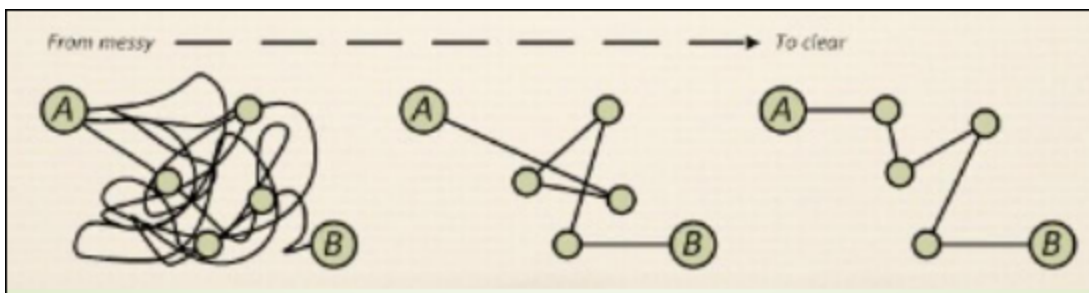
- Evitar el "deterioro del código"
- Limpiar desorden
- Simplificar el código
- Incrementar la legibilidad y la comprensibilidad
- Encontrar errores
- Reducir el tiempo de depuración
- Incorporar el aprendizaje que hacemos sobre la aplicación

- Re hacer las cosas es fundamental en todo proceso creativo

Cambios en el código

- Son cambios imperceptibles desde el exterior, no buscamos cambiar el resultado ni las entradas, pero si el camino que nos lleva de uno al otro.
- Buscamos eliminar la duplicación o complejidad innecesaria
- Mejoramos la calidad del software
- Hace que el código sea más simple y más fácil de entender
- Flexibiliza el código
- Hace que el código sea más fácil de cambiar

🧩 Proceso de refactoring



Ejemplo practico

Taxis libres (Identificador verde) con chapa valida para un barrio existente

```
def test_nombre_us_camino_resultado():
```

```
def test_mostrar_mapa_taxis_libres_en_barrio():
```

```
usuario = {'nombre': 'Franco'
```

```
'apellido': 'Colapinto',  
'permisos': 'Admin de la central'}  
usuario =  
iniciar_sesion('admin', 'clave_admin')  
configurar_software('ciudad de Córdoba')  
taxis = [{'chapa': 'ABC123',  
'estado': 'Libre',  
'barrio': 'Nueva cordoba',  
'ubicacion': (10, 20)}]
```

Definir pasos del caso de prueba

```
seleccionar_opcion_menu('Ver mapa de taxis')  
mostrar_mapa(opcion: 'Mostrar taxis por estado y por barrio', barrio= 'Nueva  
cordoba', estado: 'Libre')
```

Resultados

assert: Palabra clave que se usa para verificar condiciones en las pruebas. Si la condicion dentro del assert es falsa, la prueba fallará

assert all: implica que se deben cumplir todas las condiciones para que cumpla la prueba

```
mapa_mostrado = mostrar_mapa() assert mapa_mostrado['ciudad'] == 'Cordoba',  
'El mapa mostrado no corresponde a la ciudad de Cordoba'  
  
assert all (taxi['estado'] == 'Libre')  
for taxi in mapa_mostrado['taxis']:  
    "NO todos los taxis estan en estado libre"  
  
assert all (taxi['icono'] == 'Verde')  
for taxi in mapa_mostrado['taxis']:  
    "NO todos los taxis tienen el icono en color verde"
```